

# G1 Robot — Test Protocol

## Comparing DCE meta-reasoning vs the FSM baseline (deployment-time capability evaluation)

Step-by-step procedure to run the robot experiments for the paper's evaluation section: the DCE meta-reasoner, the reactive FSM baseline, and the four ablations, across the four deployment conditions. Each run logs everything for offline comparison.

**Read Part 1 (Safety) first.** A spotter holds the remote (L2+B = stop) for every run. Use water, never hot liquid. Battery > 80% except the deliberate low-battery condition.

### The three governance modes you compare

- **DCE** (our meta-reasoner): governs by experience — switches analogy (efficient / cautious / payload-sensitive / human-aware). Branch feature/escmr-meta-reasoning, enabled with G1\_META=1.
- **FSM baseline** (the paper's comparator): fixed thresholds over the SAME signals. Branch feature/fsm-baseline, enabled with G1\_FSM=1.
- **Ablations**: DCE with one mechanism disabled (diagnostic). G1\_META=1 G1\_META\_ABLATE=<variant>.

# 1 · Safety

Operator holds the G1 remote at all times. **L2 + B** = damping/stop. Press it the instant anything looks wrong.

- Two people: one at the laptop, one beside the robot with the remote.
- Clear 2–3 m of floor; nobody in the path except the scripted 'human' in that condition.
- Battery > 80% (charge between runs). Low battery degrades gait and adds LiDAR noise.
- Cup rig: open cup taped low on the chest near the IMU, water to a marked line, secured so it cannot fall onto the electronics. Paper cover for the 'lid' condition.
- Stop if the robot spins in place, drifts, or loses localisation.

## 2 · Before each session (bring-up)

On the iPhone app: load the map, RELOCALISE (laser dots on the walls), camera ON, robot standing.

```
# robot link (terminal R):
ios_webkit_debug_proxy          # Ubuntu: ... -c null:9221,:9222-9322 -d
curl -s http://localhost:9221/json | head    # see the app page

# quick check:
python g1_goto.py reloccheck          # live pose + laser + camera
```

Capture the waypoints once, in this relocalised session:

```
python g1_goto.py waypoint A      # drive to A, Ctrl+C
python g1_goto.py waypoint B      # drive to B, Ctrl+C
python g1_goto.py listwp
```

(Optional) GPU perception for depth + YOLO objects + human detection:

```
# terminal P (Ubuntu GPU):
python perception_server.py --host 127.0.0.1 --port 8008 --debug \
  --depth depth_anything_v2 --seg segformer --det yolo --det-conf 0.45 \
  --fx 300 --fy 300 --cx 160 --cy 120 --cam-h 1.10 --cam-pitch -10
# then in the run terminal: export G1_PERC=127.0.0.1:8008
```

**Checkpoint:** reloccheck shows pose+laser+camera; listwp shows A and B; (if used) curl localhost:8008/health lists the GPU(s) and http://localhost:8008/ shows live video.

### 3 · How to launch each mode (to B, with the live window)

Run from a desktop session (the window needs X11). Press **ENTER** in the run terminal every time you see the cup spill — it is logged as ground truth.

#### DCE meta-reasoner

```
git checkout feature/escmr-meta-reasoning
export G1_PERC=127.0.0.1:8008 # optional (depth+YOLO)
G1_META=1 python g1_goto.py gotoviz B
```

#### FSM baseline

```
git checkout feature/fsm-baseline
export G1_PERC=127.0.0.1:8008
G1_FSM=1 python g1_goto.py gotoviz B
```

#### Ablations (DCE with one mechanism off)

```
git checkout feature/escmr-meta-reasoning
G1_META=1 G1_META_ABLATE=instantaneous python g1_goto.py gotoviz B
G1_META=1 G1_META_ABLATE=no_hysteresis python g1_goto.py gotoviz B
G1_META=1 G1_META_ABLATE=no_plausibility python g1_goto.py gotoviz B
G1_META=1 G1_META_ABLATE=no_insufficiency python g1_goto.py gotoviz B
```

If the door-centering strafe pushes the robot the **WRONG** way, relaunch with `G1_STRAFE_SIGN=-1`.  
To turn door-centering off: `G1_DOOR_CENTER=0`.

### 4 · The four conditions (how to induce each)

| Condition          | How to set it up                                                           | What should happen                                                                     |
|--------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Door crossing      | Place A and B so the path goes through the doorway (DCE: obstacle caution) | DCE: switch to <code>door_crossing</code> / aggressive at the door; FSM: threshold slo |
| Payload / no-lid   | Open cup, water to the line, NO paper cover.                               | Press ENTER on each spill. DCE: spill -> switch to <code>payload_sensitive</code>      |
| Human intervention | A person walks across ~1 m in front, mid-run (scripted)                    | DCE: switch to <code>human_aware</code> (yield/slow); FSM: <code>STOP_HUMAN</code> .   |
| Low battery        | Run deliberately at low battery (with spotter, short).                     | More LiDAR noise; check DCE avoids false switching vs FSM chatt                        |

### 5 · Suggested run order (start small, then scale)

For each condition, do paired runs: DCE first, then FSM, same layout. Begin with N=5 repeats each; scale to 10 once stable. Keep battery > 80% (except the low-battery condition).

- Door crossing: 5× DCE, 5× FSM
- Payload / no-lid (spill): 5× DCE, 5× FSM
- Human intervention: 5× DCE, 5× FSM
- Low battery: 5× DCE, 5× FSM
- Ablations (hardest condition only, e.g. payload+human): 3× each of the 4 variants

Pre-register what a 'correct switch' is BEFORE running (e.g. ordinary→cautious when clearance enters high-concern at the door; →payload\_sensitive on a spill; →human\_aware when a person is within ~1 m). Two people annotate a sample of runs

independently for agreement.

## 6 · After each run — capture and check

```
python organize_run.py          # bundles the newest run into runs/<date-time>_ours_B/  
# -> 01_trajectory.png 02_metrics.png 03_health.png run.json README.md  
python plot_trajectory.py dataset/<run>.json # path + LiDAR + YOLO objects  
python plot_metrics.py dataset/<run>.json # clearance/progression/reliability + switches  
python plot_health.py dataset/<run>.json # battery/temps/motors
```

### Quick sanity checks per run:

- Reached B? final error and #collisions (console 'LLEGADO ... err=', ncol=').
- DCE: did it switch when expected? look for META-SWITCH -> ... in goto.log and the meta\_switch events / purple line in plot\_metrics.
- FSM: how many FSM -> ... interventions (count in goto.log).
- Spills: summary.spills\_human matches what you pressed.
- Perception used? summary.perc\_queries > 0 and objects in the trajectory.

**Keep a simple sheet per run:** mode (DCE/FSM/ablation), condition, reached (y/n), time, collisions, spills, #switches/#interventions, notes. This is the raw table for the paper.

## 7 · What you are measuring (DCE vs FSM)

- **Success:** reached B without safety/task failure.
- **Appropriate switches** (DCE) vs **reactive interventions** (FSM): did it react for the right reason?
- **Unnecessary switching / chatter:** switches while the situation was fine (lower is better).
- **Payload:** spills with vs without the payload\_sensitive switch.
- **Human:** did it yield/slow appropriately?
- **Low-battery robustness:** false switches under noise (DCE should resist; FSM chatters).
- **Collisions, time, efficiency** from the summary.

## 8 · Troubleshooting

- [perc] NO RESPONDE -> perception server down or wrong IP; it still runs without depth.
- Door-centering wrong way -> G1\_STRAFE\_SIGN=-1.
- YOLO false positives (e.g. wall->refrigerator) -> raise --det-conf 0.55.
- perc\_n stays 0 (depth not seeing the table) -> camera not calibrated for 320px; use --cx 160 --cy 120 or run python calibrate\_cam.py wall.
- Robot spins / drifts -> low battery or bad relocalisation; stop (L2+B) and restart.

## 9 · Quick reference

```
# bring-up: proxy on :9221 · app relocalised · waypoints A,B · battery >80% · remote in hand  
export G1_PERC=127.0.0.1:8008  
# DCE: git checkout feature/escmr-meta-reasoning ; G1_META=1 python g1_goto.py gotoviz B
```

```
# FSM baseline:  git checkout feature/fsm-baseline          ; G1_FSM=1  python g1_goto.py gotoviz B
# ablation:      G1_META=1 G1_META_ABLATE=no_hysteresis python g1_goto.py gotoviz B
# during run:    press ENTER on every cup spill
# after run:     python organize_run.py ; plot_metrics.py / plot_health.py / plot_trajectory.py
```

Detail: [docs/SETUP\\_UBUNTU.md](#), [docs/METRICS.md](#), [docs/PERCEPTION\\_UPGRADE.md](#).